Disability Services
University of Warwick
Senate House
Coventry
CV4 8UW

Considerate Marking cover sheet

DO NOT COPY

Student ID Number: 5714960

Note to student:

It is the responsibility of the student to add this coversheet to their coursework or examinations. University staff will be unable to attach this to your work, on your behalf.

Note to assessors/markers:

This cover sheet is to prompt assessors/markers to the Specific Learning Difference (such as Dyslexia, Dyspraxia/DCD, dysgraphia, dyscalculia), and/or other condition that this student has, which is supported through the university's Reasonable Adjustments process.

Academic staff are encouraged to not unduly penalise poor spelling, punctuation, grammar or structure, unless these form explicit assessed course criteria or competency standards, but rather mark for content and demonstrated understanding.

Further information and guidance can be found here: https://warwick.ac.uk/services/wss/students/disability/how-can-we-help/reasonable-adjustments/marking/

Kind regards

Disability Services

Date issued: 02/09/2025

Competitive Analysis of Caching With Delayed Hits on Line Networks

Saatvik Kambhampati
Department of Computer Science
University of Warwick, Coventry, United Kingdom
Saatvik.Kambhampati@warwick.ac.uk

Florin Ciucu (Supervisor)
Department of Computer Science
University of Warwick, Coventry, United Kingdom
F.Ciucu@warwick.ac.uk

July 16, 2026

Abstract

Competitive analyses of caching with delayed hits (DHs) have so far been focused on single-cache systems, where a cache-miss costs a fixed constant and a cache-hit costs nothing. In industry, caches are often implemented as networks where the cost of serving a request scales with the distance it travels upstream, so hits are no longer zero-latency. This report formalises a so-called 'DH-aware' line-network model, characterised by the number of caches $|V|$, the per-cache capacity k, and the inter-cache delay Z_A. It further establishes two lower-bounds on the competitive ratio of deterministic online policies. Applying existing adversary sequences yields a trivial $\Omega(1)$, as it does not grow with network-specific parameters. This adversary is shown to be indifferent to the hit-cost that networks introduce, treating the network as a single cache. Such a result motivated the remaining work: a second adversary, tailored to the Leave-a-Copy-Everywhere (LCE) placement policy, yields $\Omega(k|V|)$. This is the first competitive bound for LCE content placement policy on a cache network, and it provides a bound that grows with the length of the network $|V|$, and the size of each cache k. Remaining work targets a matching upper-bound and simulation-based validation.

Contents

1 Introduction **3**
1.1 Cache Networks . . . 3

2 Project Problem Statement, Objectives, and Motivation **3**
2.1 The Problem . . . 3
2.2 Motivation . . . 4
2.3 Objectives . . . 4

3 Background Research **5**

4 Caching Model **6**
4.1 Formulation of Caching & Delayed-Hits . . . 6
4.2 Cache Line Networks . . . 7
4.3 Competitive Analysis for Caching . . . 7

5 Progress so Far **8**
5.1 Choosing $\mathcal{O}$ For CPPs . . . 8
5.2 Initial Baseline Lower-Bound . . . 9
5.3 Lower-Bounding LCE . . . 10

6 Conclusion **12**

7 Future Work **13**
7.1 Upper Bounds . . . 13
7.2 Timetable . . . 13

8 Project Management & Reflection **13**
8.1 Unforeseen Problems . . . 13
8.2 Appraisal and Reflection . . . 13
8.3 AI Usage . . . 14

1 Introduction

Caching stores frequently accessed data in a small, fast memory (a '*cache*') placed physically close to a receiver, reducing retrieval latency [1]; it is deployed across CPU hierarchies, Content Delivery Networks (CDNs), and networked systems [1, 2].

In caching, a client requests data from the cache first: if the item is present, the cache serves it directly (a 'hit'); otherwise (a 'miss') the request is forwarded to a *backing-store* [1], containing all possible items, which returns the item back to the client. Whether a fetched item is then admitted to the cache, and which item, if any, is evicted from the cache to make room, is decided by an *eviction policy* (sometimes called a *cache replacement policy*) [3].

The time between a request arriving and the backing-store returning a fetch for that item (assuming the request resulted in a cache-miss) is known as the *fetch time*. From this, a specific property of caching models may be defined known as the *delay parameter*. I.e. The ratio of the fetch time to the inter-request arrival time[1] [4]. This property essentially defines the number of requests that arrive at the cache while a fetch request for that same item is already en-route.

In classical caching, this delay parameter is set as zero (implying that the fetch time is zero). However, real systems may have a tangible, non-negligible lag between the cache and the backing store, such that the fetch time is not zero. This has been observed in high-throughput systems such as FPGAs [4], and in CDN hierarchies [1]. Suppose that two requests for the same item are to arrive at a cache, which does not have this item. The first request results in a cache-miss, and a fetch request is sent to the backing-store. Then, the second request arrives while this response is still in-flight. In a classical model, this results in a cache-hit as the item is instantly returned from the backing-store and admitted to the cache. However, in non-classical models, the request is placed into a queue. When the fetch for the first request is resolved, the second request is taken out of a queue and is considered a *delayed-hit* (DH). Models in which DHs are accounted for are identified as 'DH-aware' contexts or models, with the converse being 'DH-unaware' contexts.

1.1 Cache Networks

Much of what has been introduced thus far is specifically for *single-cache systems*. However, in industry, caching is often implemented in the form of *networks* (many, inter-connected caches that may query each other). In a cache network, caches forward any requests they cannot serve to an adjacent cache. In this way, cache networks can store more data over a larger physical distance. This means that they can serve geographically dispersed clients with lower latency.

2 Project Problem Statement, Objectives, and Motivation

2.1 The Problem

This project aims to extend the competitive analysis of DH-aware caching to network topologies from single-cache systems. Specifically, the behaviour of the Leave-a-Copy-Everywhere (LCE) content placement policy (CPP)[2] considered on a line network in a DH-aware context. In LCE, when the backing-store returns a fetch, the item is admitted to every cache along the

[1] The inter-request arrival time is the time between two requests for the same item

[2] A CPP is a policy that determines *to which cache* a specific item should be admitted, in contrast to an eviction policy which determines *what* should be evicted from a specific cache

queried path [5]. In this way, LCE introduces a high amount of redundancy [5]. Despite this, LCE specifically is studied here as it is heavily applied in literature as a baseline policy [5, 6] and in industry [7]. Thus, it is the best policy on which to perform an initial competitive analysis. The central contribution of researching this specific gap is to provide a tight competitive analysis of LCE on DH-aware caching networks.

In DH-unaware contexts, a cache hit incurs zero-cost, regardless of where the item is fetched from [4]. DH-aware contexts fundamentally change this. If a cache fires a miss and forwards a fetch request, and the next cache in the network fires a hit, the delay incurred is that between two adjacent caches. This is less than the latency of forwarding to a deeper node, or to the backing-store. Thus, hit cost is now proportional to the 'fetch distance'. This provides an interesting problem to be solved in the context of competitive analysis as to the tightness guarantees of LCE on a network topology.

2.2 Motivation

Caching is the backbone of many modern networks, such as CDN edge servers providing quick access to web content [2]. In such systems, the delay between a cache and the origin server (backing-store) is non-negligible, meaning DHs become a real, common phenomenon. A CDN edge cache that fetches data from a 'regional hub' incurs a fundamentally different cost to one that must fetch it from the origin (and further, during this fetch time, every subsequent request for that data is halted).

Despite this, existing competitive analyses of DH-aware caching consider only isolated single-cache systems, wherein fetch distance is constant. In many real scenarios, caches can be implemented as networks in a hierarchical or linear topology, where the cost of a miss depends on how 'far' the fetch request has travelled. Understanding how CPPs such as LCE interact with DHs on such topologies is therefore a necessary step toward providing guarantees for the networked systems in which caching is actually deployed.

2.3 Objectives

The objectives for this project are as follows:

1. Formally define the DH-aware line caching model, including the cost function and the LCE placement policy;
2. Construct an adversarial request sequence establishing a lower-bound;
   (a) Establish a baseline lower-bound using Manohar & Williams's adversarial sequence [8], which will help guide future work;
   (b) Establish the lower-bound for LCE;
3. Prove an upper bound on the competitive ratio (CR)[3] of LCE on a line of caches as a function of the number of caches, size of each one, and the delay parameter, thereby showing tightness;
4. Develop a DH-aware cache network simulator capable of modelling line topologies under configurable policies, cache sizes, and delay parameters;

[3]The CR is a metric to evaluate the performance of algorithms in comparison to their optimum counterparts, and is introduced in more detail in Section 4.3

5. Validate the theoretical bounds experimentally by comparing simulated CRs against the proven guarantees; and

6. Investigate generalisations to tree topologies as extensions.

This report discusses objectives 1, 2(a), and 2(b); Section 7 outlines the plan for objectives 3–6.

3 Background Research

Atre et al. [4], in 2020, highlighted the effect that DHs could have on caching systems. They show that, by assuming the existence of a non-negligible fetch time, a request for an item that arrives during an existing fetch request for that item should not be classified as a cache-miss or a cache-hit. Atre et al. argues that neither is 'correct'. Instead, they propose a third classification: the delayed-hit. They argue that the cost incurred by this second request is in fact less than that of the first (because the second request 'waits' less time for the first request to be fulfilled). Thus, they classify it as a *delayed hit*. They show the sub-optimality of the so-called 'optimal' offline algorithm, MIN (Belády's [9]), when observed in a DH-aware context. Furthermore, they discuss the discrepancy between hit ratio and latency, whilst providing an alternative DH-aware metric: the Aggregated Delay. They then propose a new latency-optimal offline algorithm known as BELATEDLY. Finally, they introduce a novel online algorithm for DH-aware caching known as MINIMUM-AGGREGATEDELAY.

In May 2020, Manohar and Williams [8] published a paper that proposed a competitive ratio (CR) lower-bound for any deterministic DH-aware online caching algorithm. Specifically, they find that the lower-bound grows with both the capacity of the cache and the delay parameter. In the same publication, they further prove that Atre et al.'s DH model did not exhibit anti-monotonicity and provide a relevant modification to their work and the model. Anti-monotonicity is a valuable property to retain in caching as, intuitively, a cache hit should result in a lower latency than a cache miss. Their modification is to simply fetch an item from the backing store, *even* when the request results in a cache-hit, resulting in a model that never 'rewards' a miss more than a hit.

In 2025, Gurushankar et al. [10] prove that the LEAST-RECENTLY-USED (LRU) eviction policy, in a DH-aware context, has a CR upper-bound that *also* scales with the cache capacity and the delay parameter. The two bounds 'match', meaning that this provides the first 'tight guarantee' for delayed-hits caching with the LRU. They prove that this guarantee extended to any marking algorithm. That year, Tan et al. [11] also published a paper that showed any randomised caching algorithm has a CR lower-bound that scales with the cache capacity and the logarithm of the delay parameter ($\Omega(k \log Z)$). They then provide the outline for a novel randomised caching algorithm 'CaLa', proving the upper-bound to be $O(k \log Z)$-competitive, thereby matching the bounds and providing a tight guarantee.

Separately, placement strategies for caching networks have been studied in DH-unaware contexts. Strategies such as LCE and LEAVE-A-COPY-DOWN (LCD) move content toward the client on cache hits [6]. An alternative approach, DEMOTE [7, 12], instead pushes evicted items one hop upstream in the cache hierarchy on replacement, preserving recently-evicted content deeper in the network. While LCE is very prevalent in the literature, it has not been analysed under the DHs model or subjected to competitive analysis.

4 Caching Model

4.1 Formulation of Caching & Delayed-Hits

Using the notation in Table 1, the DH-unaware (classical) context would be when the delay parameter $Z = 0$, whereas the DH-aware context is $Z \geq 1$. This notation will be used throughout the rest of the report.

Table 1: Notation for the formal caching model.

Symbol	Type / Domain	Definition
$\mathcal{I}$	finite set	The universe of items.
T	$T \subset \mathbb{R}_{\geq 0}$	Discrete set of request times.
r	$r = (i, t) \in \mathcal{I} \times T$	A single *request* for item i at time t.
ρ	$\rho = (r_j)_{j=1}^{N}$	Request sequence, time-ordered: $t_1 \leq \cdots \leq t_N$.
B	$B \subseteq \mathcal{I}$	Backing-store.
k	$k \in \mathbb{N}$	Cache *capacity*.
C_t	$C_t \subseteq \mathcal{I} \times T$, $\lvert C_t \rvert \leq k$	Cache state at time t; abbreviated C when unambiguous.
τ	$\tau \in T$	Policy-dependent timestamp of a cached item (e.g. insertion time, or time-until-eviction).
$h(i, C_t)$	$\{0, 1\}$	Hit indicator, $\mathbf{1}[i \in \mathrm{dom}(C_t)]$; $h = 1$ is a *hit*, $h = 0$ a *miss*.
t_b	$t_b \in \mathbb{R}_{\geq 0}$	Time at which B fulfils a fetch issued at t.
F	$F = t_b - t \ \ (\geq 0)$	Fetch time.
R	$R > 0$	Inter-request arrival time.
Z	$Z = \frac{F}{R}$	*Delay parameter*: the number of requests for i arriving during one fetch for i [4].
i_{ev}	$i_{\mathrm{ev}} \in \mathrm{dom}(C_t)$	Victim item chosen by the eviction policy.
update	$(\mathcal{I} \times T) \times 2^{\mathcal{I} \times T} \to 2^{\mathcal{I} \times T}$	State transition $C_t \mapsto C_{t+1}$: $\big(C_t \setminus \{(i_{\mathrm{ev}}, \tau_{\mathrm{ev}})\}\big) \cup \{(i, \tau)\}$ on a miss, at $t + F$; $\big(C_t \setminus \{(i, \tau')\}\big) \cup \{(i, \tau)\}$ on a hit (refresh τ).

The advent of DH-aware models means that the time between a request arriving and being fulfilled (the *latency*) becomes a tangible, meaningful metric through which a caching system can be measured. Manohar & Williams defined latency as follows:

Definition 4.1 Latency: Given a single-cache system using a policy $\mathcal{P}$ and a sequence of requests $\rho = (r_1, r_2, r_3, \ldots, r_N)$, the latency of each request l_t is defined as:

$$l_t = (1 - h(i))(Z - (t - p_t)),$$

where p_t is the last request for the same item i requested at time t that resulted in a *cache-miss*.

4.2 Cache Line Networks

Figure 1 shows a general line topology cache network $\mathcal{N} = (V, E)$. Every vertex $v_i \in V$ represents a cache $C_t^{(i)}$, and is connected to exactly one other vertex $v_j \in V$ (i.e. $(v_i, v_j) \in E$) or to B. Each cache $C_t^{(i)}$ has a capacity $|C_t^{(i)}| \leq k$. Further, let N be the set of all items stored in $\mathcal{N}$. When a request for an item arrives at some cache v_i, if a miss fires, then the request is sent upstream to cache v_{i+1}. In this way, a network can be thought of as being made of multiple cache-backing-store pairs, connected in series. This justifies the introduction of the *inter-cache delay parameter*, Z_A. That is, the delay that exists between two connected caches. Further, the delay parameter between v_0 and B is denoted as $Z_B = |V|Z_A$, and the total capacity of the network is $k_T = |V|k$.

Cache-networks, as a whole, are governed by *content placement policies* (CPPs), also known as *decision policies* [7]. This is similar to how single-cache systems are governed by *eviction policies*. The CPP determines *which* cache a given item is admitted to [7]. Then, that cache's specific eviction policy is triggered to determine which item the cache will evict in order to admit i. Common CPPs include DEMOTE and LCE [7].

Finally, the process for a request r_k arriving at a cache-network can be defined as follows [13]:

1. Request r_k for item $i \in \mathcal{I}$ arrives at time t to cache-vertex v_0.
2. If $h(i) = 1$ (hit) then v_0 returns the item, incurring latency $l_k = 0$.
3. If $h(i) = 0$ (miss), then v_0 forwards the request upstream, incurring a latency of $l_k = Z_A$.
4. Repeat from step 2. Each time, $l_k \leftarrow l_k + Z_A$ until B is reached or $h(i) = 1$, at which point $l_k = d \cdot Z_A$, where d is the number of hops made. If B is reached, then $l_k = d \cdot Z_A = |V|Z_A = Z_B$. B then returns item i to v_0. The eviction policy and CPP are applied to determine the redistribution of items across the caching network to allow v_0 to admit i.

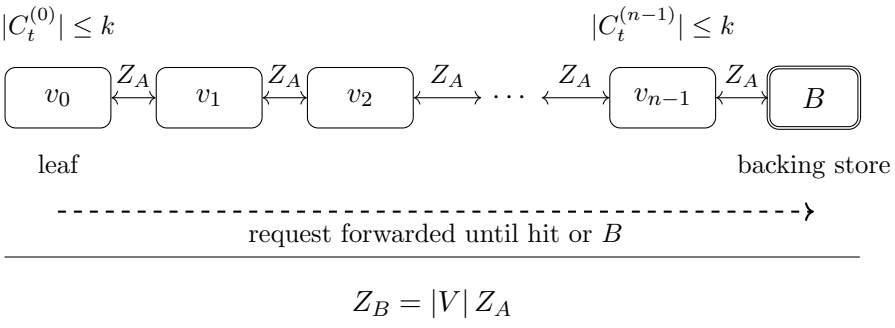


Figure 1: A line topology of $|V|$ caches

4.3 Competitive Analysis for Caching

With the caching, the line topology, and latency now defined, the central metric of this project may be formalised: the *competitive ratio* (CR). This measures an online policy [4] against the optimal offline policy [5] on a DH-aware line network.

[4] An online policy is one that only knows past and current requests.

[5] An offline policy is one that can see the full request sequence, including future requests. Non-trivial offline policies are optimal ones.

Definition 4.2 Competitive Ratio - [10]: Given an online caching policy $\mathcal{P}$ and an offline optimal policy $\mathcal{O}$, the CR of $\mathcal{P}$ in a delayed-hits-aware context ($Z \geq 1$) is

$$\mathrm{CR}(\mathcal{P}) = \sup_{\rho} \frac{L(\mathcal{P}, \rho)}{L(\mathcal{O}, \rho)},$$

where $L(\mathcal{P}, \rho) = \sum_{r_j \in \rho} l_j$ (Def 4.1) is the total latency incurred by $\mathcal{P}$ on $\rho = (r_j)_{j=1}^{N}$.

It is ideal to obtain CR = 1 as this implies that $\mathcal{P}$ is as good as $\mathcal{O}$ for the sequence ρ, with lower CRs being more optimal.

In 2020, Manohar and Williams, proved the theoretical lower-bound for the CR of any deterministic policy acting in a DH-aware context. A CR lower-bound is defined as follows:

Theorem 4.1 Competitive Ratio Lower-Bound - [8, 14]: Let $\mathcal{P}$ be any deterministic online caching policy. Then there exists a request sequence ρ_A such that

$$\frac{L(\mathcal{P}, \rho_A)}{L(\mathcal{O}, \rho_A)} \geq c \cdot x \implies CR(\mathcal{P}) \geq c \cdot x,$$

for some constant $c > 0$ and a variable parameter x. Equivalently,

$$\mathrm{CR}(\mathcal{P}) = \Omega(x).$$

Then, their subsequent lower-bound is defined as follows:

Theorem 4.2 Manohar & Williams's Lower-Bound – [8]: In a single-cache system, given a deterministic online eviction policy $\mathcal{P}$ and an offline eviction policy $\mathcal{O}$, the competitive ratio of $\mathcal{P}$ has a lower-bound of

$$\mathrm{CR}(\mathcal{P}) = \Omega(kZ),$$

where k is the size of the cache, and Z is the delay parameter.

5 Progress so Far

This section presents two novel contributions: a baseline lower-bound obtained by applying the Manohar–Williams sequence ρ_{MW} to a line network, and a lower-bound for the LCE placement policy. Both are preceded by the restrictions placed on $\mathcal{O}$, which are non-standard for CPPs.

5.1 Choosing $\mathcal{O}$ For CPPs

In literature, $\mathcal{O}$ represents the optimal, offline, *eviction* policy. These are Belády's MIN algorithm in classical caching [9], and the BELATEDLY algorithm for DH-aware caching [4]. It typically has the ability to evict whichever item it 'wants' to after each cache-miss. However, here, $\mathcal{O}$ represents the optimal, offline CPP [5]. It *cannot* determine which items are evicted from individual caches, as this is determined by the eviction policy (TIME-TO-LIVE (TTL) here). So, $\mathcal{O}$ should be able to freely place any item that B returns, and any item evicted by any cache, to any cache at its discretion.

Optimal eviction policies implicitly gain admission policy properties. They may admit an undesirable item and then evict it immediately, reverting the cache to its previous state. An optimal CPP, however, retains no such option because eviction is not under its control. Accordingly, $\mathcal{O}$ is granted the ability to decline placement outright.

5.2 Initial Baseline Lower-Bound

The Manohar–Williams sequence ρ_{MW} establishes $\Omega(kZ)$ on a single cache. Applying this sequence to a line network is the natural first step: it tests whether the existing adversary is sensitive to topology, and so provides a baseline that can guide future work. First, initialise the cache network $\mathcal{N}$ with the first k_T initial items in $\mathcal{I}$:

$$\begin{aligned} C_0^{(0)} &= \{i_1, i_2, \ldots, i_k\}, \\ C_0^{(1)} &= \{i_{k+1}, i_{k+2}, \ldots, i_{2k}\}, \\ &\ldots, \\ C_0^{(|V|-1)} &= \{i_{(|V|-1)k+1}, i_{(|V|-1)k+2}, \ldots, i_{k_T}\} \end{aligned}$$

Next, define ρ_{MW} within the context of a *cache network*:

1. Make a pure request for $i \in \mathcal{I}$ s.th. $i \notin N$ (that is, i is not in any cache in the network).
2. Make a bursty request [6] for $i \in \mathcal{I}$ s.th. $i \notin N$.
3. Repeat step 2 $k_T - 1$ more times.

Analysis for $\mathcal{P}$. Request r_1 (pure) results in an initial miss, as $i \notin N$, incurring $l_1 = Z_B$. Then, each subsequent bursty request makes a request for whichever item is also not in N. Since each bursty request has Z_B actual requests, each with a latency of $Z_B - p + 1$ (as they all miss), each bursty request has a latency of

$$\sum_{p=1}^{Z_B} (Z_B - p + 1) = \frac{Z_B(Z_B + 1)}{2}$$

And, since there are k_T-many bursty requests:

$$L(\mathcal{P}, \rho_{\text{MW}}) \geq Z_B + k_T \cdot \frac{Z_B(Z_B + 1)}{2}$$

This is precisely the result of Manohar & Williams ($L(\mathcal{P}, \rho_a) \geq Z + kZ\frac{Z+1}{2}$) [8], under the reparametrisation:

- $k \mapsto k_T$; and
- $Z \mapsto Z_B$.

[6] A bursty request is $(\perp^Z, i^Z, \perp^Z)$ where $\perp$ is no request and i^Z means Z requests for item i.

Analysis for $\mathcal{O}$. Again, request r_1 results in an initial miss. Like in the single-cache system, $\mathcal{O}$ cannot avoid this miss, as it can only re-arrange on the return of a fetch request. This miss causes $\mathcal{O}$ to incur $l_1 = Z_B$. Then, as in the single-cache analysis, $\mathcal{O}$ re-arranges the cache network once, evicting the one item that is never requested, thus resulting in a 100% hit-rate for all bursty requests.

However, the key difference here is that hits do not necessarily incur zero latency. Every hit incurs a latency of $d \cdot Z_A$ where d is the number of caches that were queried before either hitting or reaching B. And, since after r_1, every item requested is an item that is already stored in N [8], meaning that each item (k in total) in each cache ($|V|$ in total) is requested, so the latency incurred by $\mathcal{O}$ on all bursty requests is:

$$L_B(\mathcal{O}, \rho_{\text{MW}}) = \underbrace{Z_B}_{\text{Bursty}} kZ_A \sum_{d=1}^{|V|-1} d = kZ_B Z_A \cdot \frac{(|V|-1)|V|}{2}$$

Using $Z_B = |V|Z_A$:

$$L(\mathcal{O}, \rho_{\text{MW}}) = kZ_B^2 \cdot \frac{(|V|-1)}{2}$$

Computing the CR in closed form. The competitive ratio may thus be found as:

$$\text{CR}(\mathcal{P}) \geq \frac{2 + k|V|^2 Z_A + k|V|}{2 + k|V|^2 Z_A - k|V|Z_A}$$

As $|V|, k, Z_A \to \infty$:

$$\text{CR}(\mathcal{P}) \geq \frac{k|V|^2 Z_A}{k|V|^2 Z_A - k|V|Z_A} = \Omega(1)$$

Evaluation of Result. The sequence constructed by Manohar & Williams does not generalise to a cache network. The resulting bound is $\Omega(1)$, which is trivial by definition. This contrasts sharply with the single-cache case, where the same sequence yields $\Omega(kZ)$. This can be attributed to the fact that $\mathcal{O}$ incurs latency costs as heavily as $\mathcal{P}$ during cache-hits. This is corroborated by $L(\mathcal{P}, \rho_{\text{MW}})$, which is identical to the single-cache result. The adversary treats the network as one large cache, rather than as multiple inter-connected ones.

5.3 Lower-Bounding LCE

After a fetch return from B, LCE works by leaving a copy of the fetched item at every cache along the path of queried caches [7].

Set up. Fix a line network of $|V|$ caches, each of capacity k under a TTL eviction policy, and let $W \subset \mathcal{I}$ with $|W| = k + 1$. Each cache is initialised with k distinct items of $\mathcal{I}$ in 'size' order, so that exactly k items in W reside at v_0 and one at v_1.

Example 5.1 The (2,2) Network: A line network with $|V| = 2$, $k = 2$, $Z_A = 2$ ($Z_B = 4$) and $\mathcal{I} = \{a, b, c, d, e, f\}$, $W = \{a, b, c\}$, TTL $= 6$, is used as a trace throughout this analysis. At $t = 0$, $v_0 = \{(a, 6), (b, 6)\}$, $v_1 = \{(c, 6), (d, 6)\}$.

Adversarial Sequence ρ_A. Because $|V|k < |\mathcal{I}|$ for any non-trivial cache network, there exists at least *one* $i \in \mathcal{I}$ s.th. $i \notin N$ at $t = 0$. Request this item at $t = 1$. Since $\mathcal{I} \setminus N = \{e, f\}$, requesting e results in a full miss and a fetch request (resolving at $t = 5$). If another request for e is sent at

$t = 2$ and at $t = 3$, this will incur a latency of $Z_B - 1 = 3$ and $Z_B - 2 = 2$, respectively. Then, at $t = 4$, another request for e *would* incur $Z_B - 3 = 1$, whereas a request for d (stored at $C_4^{(1)}$) incurs a latency of $Z_A = 2$. So, the first phase ($z = 1$) of ρ_A is (e, e, e, d).

At $t = 5$ the fetch for e returns. The state of the network at this time is $v_0 = \{(a, 1), (b, 1)\}, v_1 = \{(c, 1), (d, 5)\}$. LCE mandates that e is admitted at every cache, so the eviction policy evicts the item with the lowest TTL in each cache, thus providing $v_0 = \{(e, 6), (b, 1)\}, v_1 = \{(e, 6), (d, 5)\}$. Now, each element of W is continuously requested (via bursty requests) in an order such that all bursty requests incur misses. This is possible as, after $t = 5$, there is always at least one item in W that is not in the network. Thus, for this trace, the final adversarial sequence is:

$$\rho_A = (e, e, e, d, a^{Z_B}, c^{Z_B}, b^{Z_B})$$

Or, more generally:

1. Make $Z_B - 1$ pure requests for any item not in N at $t = 0$.
2. Make one pure request for any item in $C_3^{(|V|-1)}$ at $t = 4$.
3. Make a bursty request i^{Z_B} for an item $i \in \mathcal{I}$ s.th. $i \in W \wedge i \notin N$ at $t = 5$.
4. Repeat step 3 for $|W| - 1$ more times.

Analysis for $\mathcal{P}$. No policy may rearrange their caches outside of the return of a fetch request. Thus, $z = 1$ incurs:

$$\begin{aligned} l_{z=1} &= \underbrace{Z_B}_{r_1} + \underbrace{Z_B - 1}_{r_2} + \underbrace{Z_B - 2}_{r_3} + \underbrace{(|V| - 1)Z_A}_{r_4} \\ &= 3Z_B - 3 + |V|Z_A - Z_A \\ &= 4Z_B - 3 - Z_A \end{aligned}$$

Then, at $t = 5$, the network has $v_0 = \{(e, 6), (b, 1)\}, v_1 = \{(e, 6), (d, 5)\}$. Every time a new item i is fetched from B, another item i' is evicted from all $v \in V$, and i is admitted in those places. So, there are always two items $x, x' \in W$ that are not in the cache. So, every bursty request incurs misses. From Manohar & Williams, this is known to incur a latency of $\frac{Z_B(Z_B+1)}{2}$ per bursty request. Finally, since there are $k + 1$ bursty requests:

$$\begin{aligned} l_{z>1} &= (k + 1)\frac{Z_B(Z_B + 1)}{2} \\ &= \frac{Z_B(k + 1)(Z_B + 1)}{2} \\ &= \frac{kZ_B^2 + kZ_B + Z_B^2 + Z_B}{2} \end{aligned}$$

Giving:

$$\begin{aligned} L(\mathcal{P}, \rho_A) &= l_{z=1} + l_{z>1} \\ L(\mathcal{P}, \rho_A) &= 4Z_B - 3 - Z_A + \frac{1}{2}kZ_B^2 + \frac{1}{2}kZ_B + \frac{1}{2}Z_B^2 + \frac{1}{2}Z_B \\ &= \frac{1}{2}kZ_B^2 + \frac{9}{2}Z_B + \frac{1}{2}kZ_B + \frac{1}{2}Z_B^2 - 3 - Z_A \\ &= \frac{1}{2}k|V|^2Z_A^2 + \frac{9}{2}|V|Z_A + \frac{1}{2}k|V|Z_A + \frac{1}{2}|V|^2Z_A^2 - 3 - Z_A \end{aligned}$$

Analysis for $\mathcal{O}$. Like $\mathcal{P}$, $\mathcal{O}$ cannot avoid the initial misses, DHs, and hit-latencies of the $z = 1$ requests, incurring the same latency as $\mathcal{P}$:

$$l_{z=1} = 4Z_B - 3 - Z_A$$

For $z > 1$, $\mathcal{O}$ does not need to rearrange its cache network at all, as $|V|k \geq k + 1$, meaning that every item in W is already in N, at some depth, from $t = 0$. There are k hits at v_0 and then 1 hit at v_1. This holds in general as well, since $|W| = k + 1$ and N is filled with the first k_T items from $\mathcal{I}$, W will only ever request k items held in v_0 and one item in v_1. Thus, giving a latency of Z_A (due to the v_1 hits). Recalling that the v_1-hits are from a bursty request:

$$\begin{aligned} L(\mathcal{O}, \rho_A) &= l_{z=1} + l_{z>1} \\ &= 4Z_B - 3 - Z_A + Z_B Z_A \\ &= 4|V|Z_A - 3 - Z_A + |V|Z_A^2 \end{aligned}$$

Competitive Ratio.

$$\begin{aligned} CR(\mathcal{P}, \rho_A) &= \frac{L(\mathcal{P}, \rho_A)}{L(\mathcal{O}, \rho_A)} \\ &= \frac{\frac{1}{2}k|V|^2 Z_A^2 + \frac{9}{2}|V|Z_A + \frac{1}{2}k|V|Z_A + \frac{1}{2}|V|^2 Z_A^2 - 3 - Z_A}{4|V|Z_A - 3 - Z_A + |V|Z_A^2} \end{aligned}$$

As $|V|, Z_A, k \to \infty$:

$$\begin{aligned} CR(\mathcal{P}, \rho_A) &= \frac{\frac{1}{2}k|V|^2 Z_A^2}{|V|Z_A^2} \\ &= \frac{1}{2}k|V| = \Omega(k|V|) \end{aligned}$$

Evaluation of Result. The result of $\Omega(k|V|)$ is far less trivial than the result in Section 5.2, as it scales with both the per-cache capacity and the network length. The objective here was to find a bound that grew with the network-dependent parameters, and this has been partially achieved: k and $|V|$ are both present, whereas the inter-cache delay Z_A cancels, leaving the bound in terms of the delay contrast $Z_B/Z_A = |V|$ rather than the delay itself. However, although encouraging, this bound cannot be 'finalised' until a matching upper-bound is found, yielding a tight guarantee.

6 Conclusion

This report has established a DH-aware line caching network model and delivered two lower-bounds on the competitive ratio. Applying the Manohar–Williams sequence ρ_{MW} to a line network with $|V|$ caches yields a trivial lower-bound of $\Omega(1)$. This is because the network forces $\mathcal{O}$ to incur as much latency on cache-hits as $\mathcal{P}$ incurs on cache-misses, resulting in a trivial bound. Ultimately, hit-cost was an unintentional liability. This is because the adversary treats the network as a single-cache system, so its 'attack' is indifferent to the hit-cost, rather than specifically exploiting it. This motivates the second construction, tailored to LCE, which yields $\Omega(k|V|)$: the first competitive bound for LCE on a cache network, and one that grows with network-specific properties.

7 Future Work

7.1 Upper Bounds

Upper-bounds are the more technically substantial parts of the project as they are *universal* claims (i.e. the bound must hold for all ρ in the sequence space). In order to sufficiently carry out such analysis, work will first be done to thoroughly understand the techniques employed by Gurushankar et al. [10] in their upper-bound analysis for marking algorithms in the DH-aware context. Then, using such techniques, an attempted upper-bound for the LCE policy will be found.

7.2 Timetable

The remaining work is outlined and timetabled below. The consolidation and write-up can be carried out in parallel to the other objectives listed.

Deliverable	**Window**
LCE upper bound	3 Weeks
Finalise simulation & record findings	2 Weeks
Consolidation, write-up, and revision (parallel)	6 Weeks

8 Project Management & Reflection

Throughout the project, I have had regular meetings with my supervisor regarding my progress and any issues that have arisen. I have taken his advice on board, which has allowed me to focus my efforts efficiently.

8.1 Unforeseen Problems

One major unforeseen problem occurred when implementing $\mathcal{O}$. The simulator is intended as a generalised framework, able to model any cache topology, which requires $\mathcal{O}$ to be implemented generally — applicable to line and tree topologies alike. Such an algorithm is difficult to construct and outside the scope of this project. The specific instance of $\mathcal{O}$ for line topologies under LCE and TTL was therefore implemented instead. This still allows for empirical verification of the theoretical findings, but trades applicability for tractability.

8.2 Appraisal and Reflection

Progress has, on the whole, kept in line with the original timetable: the lower-bound analysis was completed on schedule, adequately setting up the start of the upper-bound analysis. Consistent with the theoretical focus of this project, the simulator was prioritised as a validation tool rather than a contribution. The remaining work is to measure how the competitive ratio grows with the pre-established parameters. This should be completed shortly after this report's submission, allowing a final consistency check of the lower-bound before moving to the upper-bound.

The core lesson thus far has been approaching the analysis and lower-bounding efficiently. For some days, the approach taken was trial and error to find the adversarial sequence: observing what works and doesn't work with one sequence, and iterating it to attempt to provide a 'better' one. This was inefficient, however. The approach was then changed: existing literature was examined for common techniques and patterns used, which were then applied to find a non-trivial lower-bound.

8.3 AI Usage

In this project, I used AI (specifically Claude Opus 4.6 and 4.8) to challenge and scrutinise my ideas and analyses. I conversed with it when analysing the network's behaviour, asking it to challenge my ideas and find gaps in my logic. This aided my understanding and work in two ways. Firstly, it forced me to defend my methods many times, deepening my understanding of core concepts. Secondly, it allowed me to quickly determine which ideas would work and which would not. I used it as a colleague with whom I discussed my research, as opposed to a tool to conduct my research on my behalf. The adversarial constructions, the latency analysis, the arguments, and the simulator are my own work.

Secondly, I used AI as a spell and grammar checker in writing this report. I also used it to correct my phrasing where appropriate.

References

[1] B. Jiang and C. Ma, "Modeling and optimizing latency for delayed hit caching with stochastic miss latency," *arXiv preprint arXiv:2505.15531*, May 2025. [Online]. Available: https://arxiv.org/abs/2505.15531

[2] Cloudflare, "What is an edge server?" https://www.cloudflare.com/en-gb/learning/cdn/glossary/edge-server/, n.d., accessed: 23 February 2026.

[3] A. Jain, C. Lin, N. Jerger, and M. Martonosi, *Cache Replacement Policies*, ser. Synthesis Lectures on Computer Architecture. Morgan & Claypool Publishers, 2019. [Online]. Available: https://books.google.co.uk/books?id=QRieDwAAQBAJ

[4] N. Atre, J. Sherry, W. Wang, and D. S. Berger, "Caching with delayed hits," in *Proceedings of the ACM SIGCOMM*. ACM, 2020, pp. 495–510.

[5] S. Ioannidis and E. Yeh, " Adaptive Caching Networks With Optimality Guarantees ," *IEEE/ACM Transactions on Networking*, vol. 26, no. 02, pp. 737–750, Mar. 2018. [Online]. Available: https://doi.ieeecomputersociety.org/10.1109/TNET.2018.2793581

[6] A. Ioannou and S. Weber, "A survey of caching policies and forwarding mechanisms in information-centric networking," *IEEE Communications Surveys & Tutorials*, vol. 18, no. 4, pp. 2847–2886, 2016.

[7] G. Zhang, Y. Li, and T. Lin, "Caching in information centric networking: A survey," *Computer Networks*, vol. 57, no. 16, pp. 3128–3141, 2013, information Centric Networking. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1389128613002235

[8] P. Manohar and J. Williams, "Lower bounds for caching with delayed hits," *arXiv preprint arXiv:2006.00376*, 2020. [Online]. Available: https://arxiv.org/abs/2006.00376

[9] L. A. Belady, "A study of replacement algorithms for virtual-storage computer," *IBM Syst. J.*, vol. 5, pp. 78–101, 1966. [Online]. Available: https://api.semanticscholar.org/CorpusID:8344798

[10] K. Gurushankar, N. G. Singer, and B. Subercaseaux, "Latency guarantees for caching with delayed hits," *Proceedings of IEEE INFOCOM*, 2025.

[11] H. Tan, Y. Wang, C. Zhang, G. Li, H. Du, Z. Han, S. H.-C. Jiang, and X.-Y. Li, "Asymptotically tight approximation for online file caching with delayed hits and bypassing," *IEEE Transactions on Networking*, vol. 33, no. 4, pp. 1886–1899, 2025.

[12] T. Wong and J. Wilkes, "My cache or yours? making storage more exclusive," in *2002 USENIX Annual Technical Conference (USENIX ATC 02)*. Monterey, CA: USENIX Association, Jun. 2002. [Online]. Available: https://www.usenix.org/conference/2002-usenix-annual-technical-conference/my-cache-or-yours-making-storage-more-exclusive

[13] D. S. Berger, P. Gland, S. Singla, and F. Ciucu, "Exact analysis of ttl cache networks," *Performance Evaluation*, vol. 92, pp. 79–91, 2014.

[14] A. R. Karlin, M. S. Manasse, L. Rudolph, and D. D. Sleator, "Competitive snoopy caching," *Algorithmica*, vol. 3, no. 1, pp. 79–119, Nov. 1988. [Online]. Available: https://doi.org/10.1007/BF01762111